RAG Architecture Notes: Page 1

Retrieval-Augmented Generation (RAG) combines a search step with a language model. Instead of relying only on what the model memorised during training, RAG retrieves relevant passages from your own documents and supplies them as context to the language model.

A RAG system has two pipelines. The indexing pipeline prepares documents ahead of time. The question-answering pipeline runs every time a user asks a question.

The Indexing Pipeline: Page 2

Step 1, Upload: the user provides PDF, Excel, Word or text files. Step 2, Extract: text is read from each file together with metadata such as page numbers and sheet names. Step 3, Clean: whitespace is normalised without removing meaningful content.

Step 4, Chunk: long text is split into smaller overlapping pieces called chunks, typically a few hundred tokens each, because embedding models and prompts have size limits and smaller pieces give more precise retrieval. Step 5, Embed: each chunk is converted into an embedding, a numeric vector that represents its meaning. Step 6, Store: chunks, vectors and metadata are saved in a vector database such as ChromaDB.

Chunking versus Embeddings: Page 3

Chunking and embeddings are different steps. Chunking decides how the text is divided; it produces pieces of text. Embedding converts each piece of text into numbers; it produces vectors. Chunking happens first, and embeddings are computed for every chunk.

At question time, the question is embedded with the same model, the closest chunk vectors are retrieved from the vector database, and the chunks are passed to the language model, which writes an answer that cites its sources.